Базы данных и экспертные системы

НИЯУ МИФИ, Кафедра финансового мониторинга

Лабораторный практикум

В.Ю. Радыгин, Д.Ю. Куприянов, А.М. Малькута, Ю.С. Шаркова

Семестр 2. Лабораторная работа 5

Лабораторная работа 2-5 рассчитана на несколько занятий. В качестве задания лабораторной работы студентам предлагается создать для разработанной в лабораторной работе 2-2 структуры БД набор триггеров, представлений и функций.

Решения необходимо показать в классе преподавателю.

Общая часть задания (специальности/направления 10.X.X)

1. Для разработанной базы данных создайте триггер, контролирующий указанное в задании 0 вашего варианта условие.
2. Для разработанной базы данных напишите функцию, вычисляющую указанную в задании 1 вашего варианта информацию.
3. Для разработанной базы данных напишите агрегатную функцию, вычисляющую указанную в задании 2 вашего варианта информацию.
4. Для разработанной базы данных создайте представление и триггер, реализующие указанное в задании 3 вашего варианта условие.
5. Решите указанную в пункте 4 вашего варианта задачу.

Варианты заданий на стр. 3

Варианты для специальности/направления 10.X.X

Вариант 1-2

0. Триггер (или несколько триггеров) не позволяет удалять гонщиков, у которых количество набранных очков по результатам гран-при больше нуля, и не позволяет изменять у таких же гонщиков номер машины (если номера машин меняются от гонки к гонке, то не позволяет изменять его ни в одной из гонок). Для проверки работы триггера установите на все связи таблиц каскадное удаление.
1. По указанному идентификатору пилота функция должна вычислять его место в чемпионате. Первое место принадлежит пилоту, набравшему в сумме по результатам всех гран-при максимальное число очков. Далее по убыванию количества очков. Если у двух пилотов количество очков одинаковое, то их место вычисляется согласно номеру машины (если номер может меняться, то согласно максимальному номеру его машины): место больше у того, у кого номер меньше (в реальной жизни при равенстве очков место вычисляется путём сравнения количества побед и других результатов). Возвращает целое число.
2. Для множества строк, содержащих идентификаторы команд, вычислить среднее место, занимаемое пилотами данных команд среди всех гран-при чемпионата. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее имя и фамилию гонщика, ключевые поля таблицы гонщики, число его побед до начала сезона (не вычисленное по результатам, а хранящееся как поле) и название команды в последнем по времени проведения гран-при. Реализуйте возможность изменения числа побед гонщика через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных неограниченный по длине стек. Структура должна позволять сохранять в стек строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. push(строка) – добавление элемента (строка) на вершину стека – результат функции – сам элемент;
    b. pop() – удаление элемента с вершины стека – результат функции – сам элемент или null, если стек пуст;
    c. empty() – очистка стека – результат функции – число удаленных элементов;

d. init() – инициализация стека – создает все необходимые таблицы, удаляет старые (если стек уже создавался ранее), обнуляет последовательности – результат – null;
e. top() – просмотр вершины стека – результат – верхний элемент стека, если он есть, иначе null.

Функции никогда не приводят к ошибкам! Напишите пример работы со стеком.

Вариант 3-4

0. Триггер (или несколько триггеров) не позволяет добавлять на стеллаж больше товаров, чем количество мест в нём (если полки или места реализованы отдельно, то не позволяет добавлять и на полку и на стеллаж больше товаров, чем можно или не позволяет создавать мест более, чем можно) и не позволяет изменять его максимальную нагрузку на значение меньшее, чем суммарный вес всех хранящихся на нём товаров.
1. По указанному имени клиента и дате вычисляет количество товаров, срок договора которых истекает до переданной в качестве аргумента даты, хранящихся на складе и принадлежащих данному клиенту. Возвращает целое число.
2. Для множества строк, содержащих длину, ширину и высоту хранящихся на складе товаров, вычисляет максимальные габариты места, необходимые для того, чтобы поместился любой из хранящихся товаров, и представляет их в виде одной строки в формате «высота X ширина X длина». Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее описания клиентов, ключевые поля таблицы клиенты, и описание их товаров. Реализуйте возможность изменения описания клиентов через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных неограниченная по длине однонаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. enqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент;
    b. dequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    c. empty() – очистка очереди – результат функции – число удаленных элементов;
    d. init() – инициализация очереди – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;
    e. tail() – просмотр конца очереди – результат – последний элемент очереди, если он есть, иначе null.

Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.

Вариант 5-6

0. Триггер не позволяет создавать маршруты, на которых поезд останавливается более чем на 40 станциях.
1. По указанным двум названиям станций (или их идентификаторам, если название станции не уникально) вычисляет стоимость поездки из первой станции во вторую. Результат – число.
2. Для множества строк, содержащих идентификаторы станции, вычисляет максимальное расстояние в зонах между двумя станциями для всех возможных пар станции из данного множества. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее названия станций и их тарифных зон, ключевые поля таблицы станции и идентификаторы поездов, маршрут которых начинается на данных станциях. Реализуйте возможность изменения зон станций через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных ограниченный по длине стек. Структура должна позволять сохранять в стек строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. push(строка) – добавление элемента (строка) на вершину стека – результат функции – сам элемент, или null, если размер стека привышен;
    b. pop() – удаление элемента с вершины стека – результат функции – сам элемент или null, если стек пуст;
    c. empty() – очистка стека – результат функции – число удаленных элементов;
    d. init(размер стека) – инициализация стека указанного размера – создает все необходимые таблицы, удаляет старые (если стек уже создавался ранее), обнуляет последовательности – результат – null;
    e. top() – просмотр вершины стека – результат – верхний элемент стека, если он есть, иначе null.

    Функции никогда не приводят к ошибкам! Напишите пример работы со стеком.

Вариант 7-8

0. Триггер не позволяет добавлять каталоги более чем третьего уровня вложенности.
1. По указанному артиклю товара, возвращает общую стоимость всех заказов, включающих данный товар. Результат – число.
2. Для множества идентификаторов товаров, вычислить количество товаров, заказанных хотя бы один раз. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее названия товаров, их артикли, количество данных товаров в наличии и ключевые поля таблицы товары. Реализуйте возможность изменения в реальной таблице количества товаров в наличии через это представление.
4. При помощи таблицы и набора функций реализуйте структуру представления данных ограниченная по длине однонаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. enqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент, или null, если очередь переполнена;
    b. dequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    c. empty() – очистка очереди – результат функции – число удаленных элементов;
    d. init(размер очереди) – инициализация очереди указанного размера – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;
    e. tail() – просмотр конца очереди – результат – последний элемент очереди, если он есть, иначе null.

    Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.

Вариант 9-10

0. Триггер(ы) не позволяет добавлять в один раздел более 50 блюд и в одно блюдо более 50 ингредиентов.
1. По указанной части названия блюда, возвращает количество блюд, содержащих в названии данную подстроку. Результат – целое число.
2. Для множества идентификаторов блюд, сформировать массив названий ингредиентов, используемых для их приготовления (в виде одной строки через запятую без повторений). Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции (в теле функции массивы могут быть).
3. Создайте представление, отображающее названия блюд, ключевые поля таблицы блюда, подробную инструкцию их приготовления и названия ингридиентов. Реализуйте возможность изменения инструкции через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных неограниченное по длине множество. Структура должна позволять сохранять во множество строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. add(строка) – добавление элемента (строка) в список элементов множества – результат функции – сам элемент, или null, если такой элемент уже был;
    b. delete(строка) – удаление элемента из множества – результат функции – сам элемент или null, если такого элемента нет;
    c. empty() – очистка множества – результат функции – число удаленных элементов;
    d. init() – инициализация множества – создает все необходимые таблицы, удаляет старые (если множество уже создавалось ранее), обнуляет последовательности – результат – null;

    Функции никогда не приводят к ошибкам! Напишите пример работы с множеством.

Вариант 11-12

0. Триггер не позволяет добавлять книги с одинаковым названием на стеллажи, индексы которых отличаются более чем на 5 позиций при сортировке.
1. Позволяет по указанному авторскому указателю определить продуктивность автора. Продуктивность автора равна количеству написанных им книг. Причем, если книга написана в соавторстве, то она входит в количество не как единица, а как отношение единицы к количеству авторов книги. Результат – вещественное число.
2. Для множества идентификаторов книг, вычислить суммарное количество экземпляров, хранящихся в библиотеке. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее названия книги, количество её экземпляров, ключевые поля таблицы книги, номера стеллажей, где книга стоит. Реализуйте возможность изменения количества через это представление в реальной таблице.
4. При помощи таблицы и набора процедур (или функций) реализуйте структуру представления данных ограниченная по длине двунаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. tenqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент, или null, если очередь переполнена;
    b. henqueue(строка) – добавление элемента (строка) в начало очереди – результат функции – сам элемент, или null, если очередь переполнена;
    c. tdequeue() – удаление элемента из конца очереди – результат функции – сам элемент или null, если очередь пуста;
    d. hdequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    e. empty() – очистка очереди – результат функции – число удаленных элементов;
    f. init(размер очереди) – инициализация очереди указанного размера – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;

Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.

Вариант 13-14

0. Триггер не позволяет назначать «бесплатным» больным лечение дороже 5000 руб.
1. Позволяет по номеру карты пациента вычислить суммарную стоимость оказанных ему услуг (действий). Результат – число.
2. Для множества пациентов (номера карт), определить суммарное количество выполненных действий (обследований и т.д.). Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее ФИО больного, дату его поступления (начала периода), номер палаты (и её идентификатор, если номер не уникален), где он лежит, и ключевые поля таблицы, описывающей периоды пребывание больного. Реализуйте возможность изменения номера палаты через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных ограниченное по длине множество. Структура должна позволять сохранять во множество строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. add(строка) – добавление элемента (строка) в список элементов множества – результат функции – сам элемент, или null, если такой элемент уже был или множество переполнено;
    b. delete(строка) – удаление элемента из множества – результат функции – сам элемент или null, если такого элемента нет;
    c. empty() – очистка множества – результат функции – число удаленных элементов;
    d. init(размер множества) – инициализация множества указанного размера – создает все необходимые таблицы, удаляет старые (если множество уже создавалось ранее), обнуляет последовательности – результат – null;

    Функции никогда не приводят к ошибкам! Напишите пример работы с множеством.

Вариант 15-16

0. Триггер не позволяет указывать цену посещений тех коллекций/залов (в зависимости от варианта), в которых нет ни одного экспоната.
1. Позволяет по названию выставочного зала вычислить количество экспонатов в нём. Результат – число.
2. Для множества идентификаторов коллекций определяет минимальную стоимость билета, позволяющего посетить хотя бы одну из них. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее названия экспонатов, условия их содержания, ключевые поля таблицы экспонаты и названия коллекций, в которые экспонаты входят. Реализуйте возможность изменения условий содержания через это представление в реальной таблице.
4. При помощи таблицы и набора процедур (или функций) реализуйте структуру представления данных неограниченная по длине двунаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. tenqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент;
    b. henqueue(строка) – добавление элемента (строка) в начало очереди – результат функции – сам элемент;
    c. tdequeue() – удаление элемента из конца очереди – результат функции – сам элемент или null, если очередь пуста;
    d. hdequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    e. empty() – очистка очереди – результат функции – число удаленных элементов;
    f. init() – инициализация очереди – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;

    Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.

Вариант 17-18

0. Триггер не позволяет указывать в пересекающиеся периоды времени в одном и том же кинотеатре в одном и том же зале разные фильмы.
1. Позволяет по названию фильма найти количество кинотеатров, в которых он показывается в текущий момент времени. Результат – число.
2. Для множества названий кинокомпаний вычисляет общее количество снятых ими фильмов. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее кинотеатр, фильм, даты и времена его показа, ключевые поля сеанса. Реализуйте возможность изменения даты и времени показа фильма через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных неограниченный по длине стек. Структура должна позволять сохранять в стек строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. push(строка) – добавление элемента (строка) на вершину стека – результат функции – сам элемент;
    b. pop() – удаление элемента с вершины стека – результат функции – сам элемент или null, если стек пуст;
    c. empty() – очистка стека – результат функции – число удаленных элементов;
    d. init() – инициализация стека – создает все необходимые таблицы, удаляет старые (если стек уже создавался ранее), обнуляет последовательности – результат – null;
    e. top() – просмотр вершины стека – результат – верхний элемент стека, если он есть, иначе null.

    Функции никогда не приводят к ошибкам! Напишите пример работы со стеком.

Вариант 19-20

0. Триггер (или набор триггеров) не позволяет добавлять более 10 городов к одному туру.
1. По типу маршрута («автобус», «поезд», «авиа», «автобус/поезд», «автобус/авиа») показывать среднюю стоимость всех маршрутов данного типа (стоимость состоит из базовой стоимости и стоимости всех экскурсий на маршруте). Результат – число.
2. Для множества строк идентификаторов маршрутов вычисляет число маршрутов, посещяющих не менее 5 городов. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее дату начала тура, город выезда, город завершения тура, базовую стоимость, ключевые поля таблицы туры. Реализуйте возможность изменения базовой стоимости через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных неограниченная по длине однонаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. enqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент;
    b. dequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    c. empty() – очистка очереди – результат функции – число удаленных элементов;
    d. init() – инициализация очереди – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;
    e. tail() – просмотр конца очереди – результат – последний элемент очереди, если он есть, иначе null.

    Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.

Вариант 21-22

0. Триггер автоматически вычисляет в оплате заказа стоимость по тарифу в зависимости от километража и заполняет это поле.
1. Позволяет по указанному идентификатору водителя вычислить его продуктивность. Продуктивность вычисляется как отношение заработанных денег к пройденному километражу за все время функционирования таксопарка. Результат – число.
2. Для множества строк идентификаторов автомобилей вычисляет суммарно пройденный километраж. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее время и дату заказа, ФИО водителя, адрес подачи такси, стоимость, ключевые поля таблицы заказы. Реализуйте возможность изменения стоимости через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных ограниченный по длине стек. Структура должна позволять сохранять в стек строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. push(строка) – добавление элемента (строка) на вершину стека – результат функции – сам элемент, или null, если размер стека привышен;
    b. pop() – удаление элемента с вершины стека – результат функции – сам элемент или null, если стек пуст;
    c. empty() – очистка стека – результат функции – число удаленных элементов;
    d. init(размер стека) – инициализация стека указанного размера – создает все необходимые таблицы, удаляет старые (если стек уже создавался ранее), обнуляет последовательности – результат – null;
    e. top() – просмотр вершины стека – результат – верхний элемент стека, если он есть, иначе null.

    Функции никогда не приводят к ошибкам! Напишите пример работы со стеком.

Вариант 23-24

0. Триггер не позволяет заказывать (сущность «Информация о продаже») в одной аптеке более 20 наименований лекарств одной категории (вне зависимости от дозировки и объема упаковки).
1. Позволяет по наименованию лекарства вычислить среднее значение продаж по аптекам (относительно количества проданных упаковок). Результат – число.
2. Для множества идентификаторов лекарств вычисляет количество лекарств, купленных хотя бы один раз. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее ФИО провизора и номер аптеки (вместе с идентификатором, если номер не уникален), в которой он работает, плюс ключевые поля таблицы провизоры. Реализуйте возможность изменения номера аптеки через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных ограниченная по длине однонаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. enqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент, или null, если очередь переполнена;
    b. dequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    c. empty() – очистка очереди – результат функции – число удаленных элементов;
    d. init(размер очереди) – инициализация очереди указанного размера – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;
    e. tail() – просмотр конца очереди – результат – последний элемент очереди, если он есть, иначе null.

    Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.

Вариант 25-26

0. Триггер не позволяет отправлять письма пользователю, который удалил уже более 10 писем от данного пользователя-отправителя.
1. Позволяет вычислить полезность платных услуг для пользователя, как отношение количества полученных писем во время действия платных услуг к общему количеству полученных писем (в процентах). Результат – число.
2. Для множества логинов пользователей вычисляет количество пользователей, которые получают писем больше, чем отправляют. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее логины отправителя и получателя письма, время его отправления и ключевые поля таблицы письмо. Реализуйте возможность изменения получателя через это представление в реальной таблице.
4. При помощи таблицы и набора функций реализуйте структуру представления данных неограниченное по длине множество. Структура должна позволять сохранять во множество строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. add(строка) – добавление элемента (строка) в список элементов множества – результат функции – сам элемент, или null, если такой элемент уже был;
    b. delete(строка) – удаление элемента из множества – результат функции – сам элемент или null, если такого элемента нет;
    c. empty() – очистка множества – результат функции – число удаленных элементов;
    d. init() – инициализация множества – создает все необходимые таблицы, удаляет старые (если множество уже создавалось ранее), обнуляет последовательности – результат – null;

    Функции никогда не приводят к ошибкам! Напишите пример работы с множеством.

Вариант 27-28

0. Триггер не позволяет добавлять голы к матчу, если в результате матча счет 0 - 0.
1. Позволяет вычислить полезность игрока по его идентификатору, как соотношение разности числа голов и числа нарушений к общему числу матчей, в которых игрок играл. Результат – число.
2. Для множества идентификаторов игры получает среднее число забитых голов. Обратите внимание! Требуется агрегатная функция. Никаких массивов в аргументах функции.
3. Создайте представление, отображающее ФИО, страну и нарушения игроков. Реализуйте возможность изменения вида карточки через это представление в реальной таблице.
4. При помощи таблицы и набора процедур (или функций) реализуйте структуру представления данных ограниченная по длине двунаправленная очередь. Структура должна позволять сохранять в очередь строки, длиной не более 64 символов. Должны быть доступны следующие действия:
    a. tenqueue(строка) – добавление элемента (строка) в конец очереди – результат функции – сам элемент, или null, если очередь переполнена;
    b. henqueue(строка) – добавление элемента (строка) в начало очереди – результат функции – сам элемент, или null, если очередь переполнена;
    c. tdequeue() – удаление элемента из конца очереди – результат функции – сам элемент или null, если очередь пуста;
    d. hdequeue() – удаление элемента из начала очереди – результат функции – сам элемент или null, если очередь пуста;
    e. empty() – очистка очереди – результат функции – число удаленных элементов;
    f. init(размер очереди) – инициализация очереди указанного размера – создает все необходимые таблицы, удаляет старые (если очередь уже создавалась ранее), обнуляет последовательности – результат – null;

   Функции никогда не приводят к ошибкам! Напишите пример работы с очередью.